

Sockets

Programación Distribuida y Tiempo Real

¿Qué es un socket?

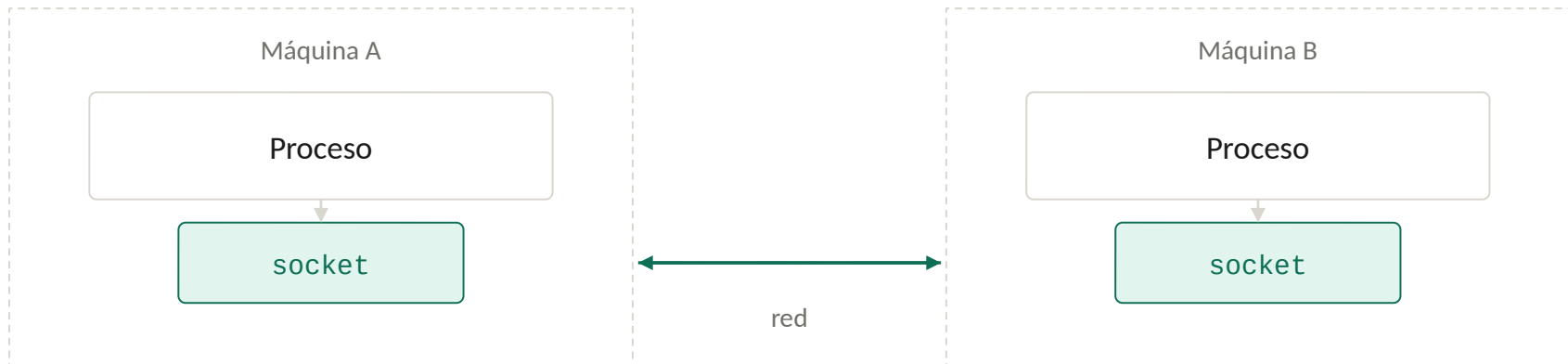
Conceptos generales

Sockets

- Mecanismo para comunicación entre procesos
- Dos procesos de una misma PC
- Dos procesos en PC diferentes unidas por la red
- A priori, un socket no implica TCP/IP para conexión
- Generalmente utiliza una estructura cliente/servidor

Conceptos generales

Dos procesos, dos extremos



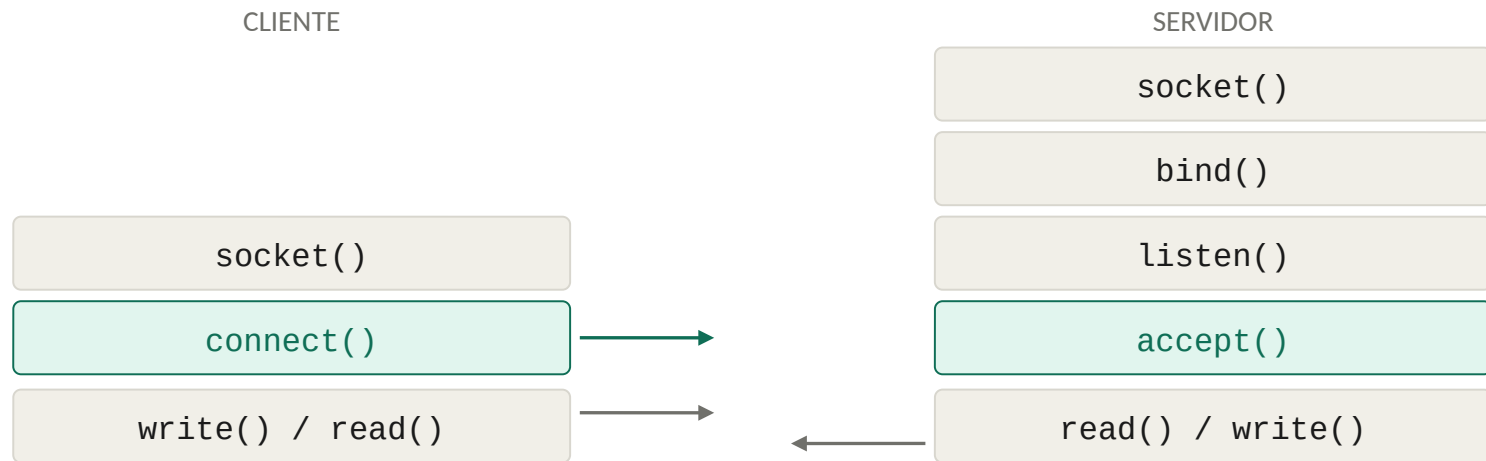
Las dos máquinas pueden ser la misma: el mecanismo no cambia.

01

Sockets en C

Sockets en C

La secuencia de llamadas



El servidor tiene que llegar hasta `listen()` antes de que el cliente pueda conectarse.

02

Cliente en C

Sockets en C

Cliente

- `socket()`

Devuelve un file descriptor del socket que luego podemos usar para llamadas al sistema. Si devuelve -1 se produjo un error.

- `connect()`

Se usa para conectarse a una dirección IP/puerto específica.

Sockets en C

Cliente

- `read()`

Lee una determinada cantidad de datos desde el socket.

- `write()`

Escribe una determinada cantidad de datos en el socket.

03

Servidor en C

Sockets en C

Servidor

- `socket()`

Devuelve un file descriptor del socket. Este socket no se conecta con el cliente: se encarga de crear las conexiones.

- `bind()`

Asocia un socket (file descriptor) con un puerto de la máquina.

Sockets en C

Servidor

- `listen()`

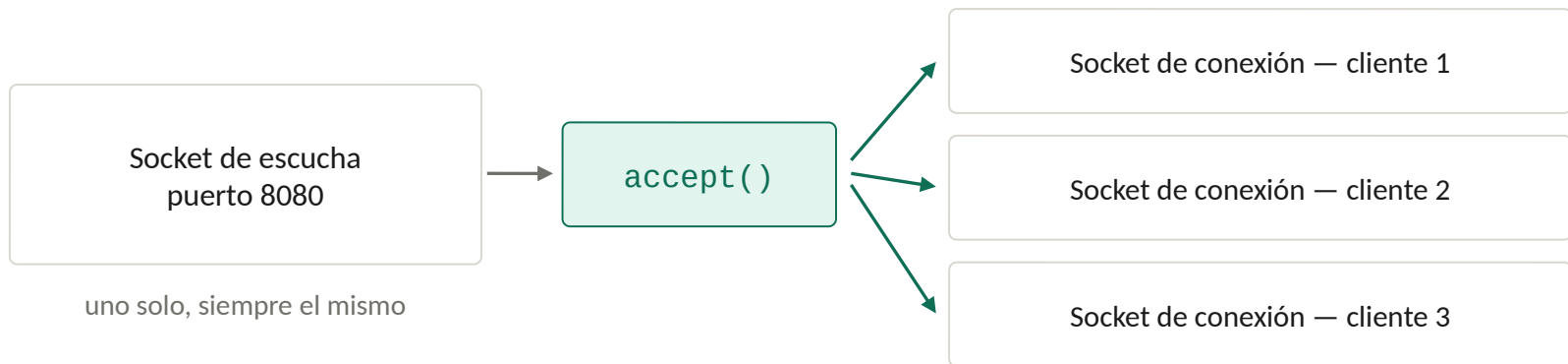
Se usa para esperar una determinada cantidad de conexiones entrantes en un socket.

- `accept()`

Se usa para esperar las conexiones desde los clientes. Devuelve un socket nuevo, que es donde se realiza la conexión. Es transparente a nosotros y es un nuevo file descriptor.

Sockets en C

Qué devuelve `accept()`



Cada llamada devuelve un file descriptor nuevo. El de escucha nunca se usa para transferir datos.

Sockets en C

Servidor

- `read()`

Lee una determinada cantidad de datos desde el socket.

- `write()`

Escribe una determinada cantidad de datos en el socket.

Ejemplo en C

Sockets en C, ejemplo comentado

linuxhowtos.org/C_C++/socket.htm

04

Sockets en Java

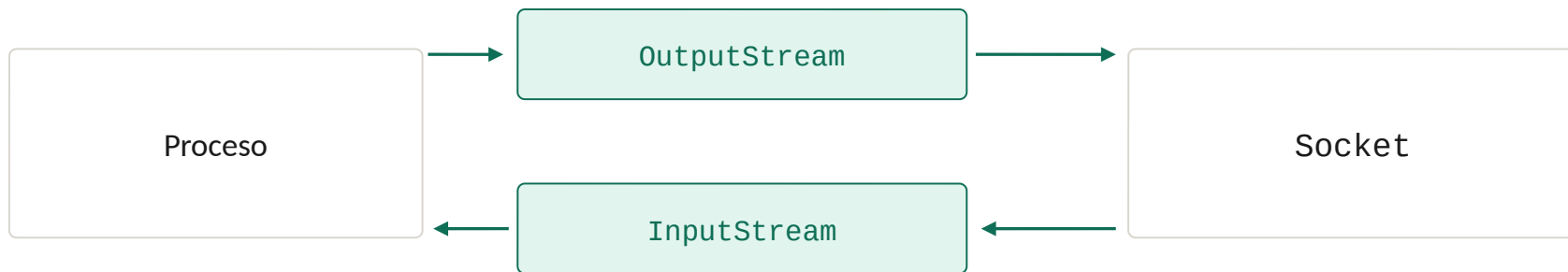
Sockets en Java

Paquetes

- Se utilizan los siguientes paquetes:
- `java.io.*`
- `java.net.*`
- Clases: `Socket()` cliente, `ServerSocket()` servidor, `InputStream`, `OutputStream`

Sockets en Java

Los streams van en el medio



En C se lee y escribe directo sobre el descriptor. En Java no: siempre a través de un stream asociado.

05

Cliente en Java

Sockets en Java

Cliente

- `Socket ( )`

Crea directamente la conexión.

Lleva como parámetros la IP y el puerto donde escucha el servidor.

Sockets en Java

Cliente

- `read()`

No lee directo desde el socket: se debe utilizar un `InputStream` relacionado al socket.

- `write()`

No escribe directo en el socket: se debe utilizar un `OutputStream` relacionado al socket.

06

Servidor en Java

Sockets en Java

Servidor

- `ServerSocket()`

Crea el socket donde se va a esperar la conexión con los clientes. Lleva el puerto donde se escucha como parámetro.

- `accept()`

Mantiene el socket a la espera de una conexión.

Sockets en Java

Servidor

- `read()`

No lee directo desde el socket: se debe utilizar un `InputStream` relacionado al socket.

- `write()`

No escribe directo en el socket: se debe utilizar un `OutputStream` relacionado al socket.

Ejemplo en Java

Sockets en Java, tutorial oficial

docs.oracle.com/javase/tutorial/networking/sockets

**¿Realmente es
cliente/servidor?**

¿Realmente es cliente/servidor?

- La conexión no implica específicamente una petición, o una respuesta a una petición: solo es comunicación
- Para ser cliente/servidor se deben cumplir los pasos de conexión: inicialización, envío/recepción de peticiones, finalización
- Protocolo de transporte:

Socket Stream — orientados a la conexión — TCP (C/S)

Socket Datagram — no orientados a la conexión — UDP

Stream y datagram

Socket Stream — TCP



conexión establecida, orden garantizado

Socket Datagram — UDP



sin conexión, cada datagrama viaja solo

El modelo cliente/servidor se apoya en el primero: hay una conexión sobre la que ocurre el intercambio.

07

Profundización

Temas relacionados

- Tipos de servidores
- Concurrencia (varios clientes)
- Seguridad en la comunicación

¡Muchas gracias!

¿Preguntas?